Universidade do Algarve

Faculdade de Ciências e Tecnologia

Licenciatura em Engenharia Informática

Análise Numérica I

Trabalho 4 – Diferenciação e Integração Numéricas

Fórmulas de Newton-Cotes Fechadas (n=1,2,3,4) e Abertas (n=0,1,2,3)

Realizado por: João Andréz nº61066, Carlos Ferreira nº 71319, Diogo Freitas nº 90147, Diogo Carvalho nº 90247

Docente: Hermenegildo Borges de Oliveira

Ano Letivo: 2025/2026

Introdução

O objetivo deste trabalho é a criação de um programa simples, em *Python*, que calcule aproximações de integrais de funções. As aproximações são calculadas usando as fórmulas de Newton-Cotes (abertas e fechadas). Para tal definimos f(x), a função a ser avaliada; e um valor *n* que, para as fórmulas abertas é o número de pontos no interior do intervalo, e para as fechadas é o número de subintervalos dentro do intervalo do integral.

Teoricamente, estas fórmulas geram aproximações com um determinado erro (inversamente proporcional ao valor de *n*) mas, neste trabalho (e para os efeitos gerais de computação), não o temos em conta.

No final do relatório encontra-se, anexado, testes da utilização do programa para cada uma das fórmulas, em que $f(x) = \sin(x)$ e o intervalo de integração é $[0, \frac{\pi}{4}]$.

Código

```
# TRABALHO 4 - Diferenciação e integração numéricas
# João Andréz - 61066
# Carlos Ferreira - 71319
# Diogo Freitas - 90147
# Diogo Carvalho - 90247

from sympy import *

def parse_value(value_str):
    """Passar string como valor avaliado pelo"""
    try:
        value_str = value_str.strip().lower()
        expr = sympify(value_str)
        return float(expr.evalf())
    except:
        raise ValueError(f"Cannot parse value: {value_str}")

def verify_interval(interval):
    """Verifica o intervalo a, b"""
    try:
        a = parse_value(interval[0])
        b = parse_value(interval[1])
        return a < b
    except (ValueError, IndexError):
        return False

def evaluate_function(f, x_val):
    """Avalia a função num ponto específico"""
```

```
    return f.evalf(subs={"x": x_val})

def open_newton_cotes(f, a, b, n):
    match n:
        case 0:
            # Regra do Ponto Médio
            h = (b - a) / 2
            x0 = a + h
            fx0 = evaluate_function(f, x0)
            return 2 * h * fx0

        case 1:
            h = (b - a) / 3
            x0 = a + h
            x1 = a + 2 * h
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            return (3 * h / 2) * (fx0 + fx1)

        case 2:
            # Regra de Milne
            h = (b - a) / 4
            x0 = a + h
            x1 = a + 2 * h
            x2 = a + 3 * h
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            fx2 = evaluate_function(f, x2)
            return (4 * h / 3) * (2 * fx0 - fx1 + 2 * fx2)

        case 3:
            h = (b - a) / 5
            x0 = a + h
            x1 = a + 2 * h
            x2 = a + 3 * h
            x3 = a + 4 * h
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            fx2 = evaluate_function(f, x2)
            fx3 = evaluate_function(f, x3)
            return (5 * h / 24) * (11 * fx0 + fx1 + fx2 + 11 * fx3)

def closed_newton_cotes(f, a, b, n):
    match n:
        case 1:
            # Regra do Trapézio
            h = (b - a) / 1
            x0 = a
            x1 = b
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            return (h / 2) * (fx0 + fx1)

        case 2:
```

```
            # Regra de Simpson
            h = (b - a) / 2
            x0 = a
            x1 = a + h
            x2 = b
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            fx2 = evaluate_function(f, x2)
            return (h / 3) * (fx0 + 4 * fx1 + fx2)

        case 3:
            # Regra de Simpson 3/8
            h = (b - a) / 3
            x0 = a
            x1 = a + h
            x2 = a + 2 * h
            x3 = b
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            fx2 = evaluate_function(f, x2)
            fx3 = evaluate_function(f, x3)
            return (3 * h / 8) * (fx0 + 3 * fx1 + 3 * fx2 + fx3)

        case 4:
            # Regra de Boole
            h = (b - a) / 4
            x0 = a
            x1 = a + h
            x2 = a + 2 * h
            x3 = a + 3 * h
            x4 = b
            fx0 = evaluate_function(f, x0)
            fx1 = evaluate_function(f, x1)
            fx2 = evaluate_function(f, x2)
            fx3 = evaluate_function(f, x3)
            fx4 = evaluate_function(f, x4)
            return (2 * h / 45) * (7 * fx0 + 32 * fx1 + 12 * fx2 + 32
* fx3 + 7 * fx4)

def main():
    print("=" * 50)
    print("Diferenciação e integração numéricas - Fórmulas de Newton-
Cotes")
    print("=" * 50)

    while True:
        expression = input("Introduza a função (f(x)): ")

        try:
            expression = sympify(expression)
        except:
            print("Erro ao processar a função. Tente novamente.")
            continue
```

```
        while True:
            interval = input("Insira o intervalo (a b): ").split()

            if verify_interval(interval):
                print("\nSelecione as fórmulas a usar:")
                print("1. Fórmulas de Newton-Cotes Abertas")
                print("2. Fórmulas de Newton-Cotes Fechadas")
                print("0. Sair")

                method_choice = input("Escolha (0/1/2): ").strip()
                while method_choice not in ("0", "1", "2"):
                   method_choice = input("Escolha (0/1/2): ").strip()
                break
            else:
                print("Intervalo inválido! Verifique que a < b.")

        if method_choice == "0":
            print("Xau :)")
            break

        while True:
            try:
                n = int(input("Insira o valor de n: "))
                if method_choice == "1" and n not in [0,1,2,3]:
                     print("O valor de n para as Fórmulas de Newton-
Cotes Abertas tem de ser 0, 1, 2 ou 3.")
                elif method_choice == "2" and n not in [1,2,3,4]:
                     print("O valor de n para as Fórmulas de Newton-
Cotes Fechadas tem de ser 1, 2, 3 ou 4.")
                else:
                    break
            except ValueError:
                print("Por favor, insira um número válido.")

        a = parse_value(interval[0])
        b = parse_value(interval[1])

        if method_choice == "1":
            result = open_newton_cotes(expression, a, b, n)
            print(f"\nAproximação do integral: {result}")
        else:
            result = closed_newton_cotes(expression, a, b, n)
            print(f"\nAproximação do integral: {result}")

              retry  =  input("\nResolver  outra  função?  (s/n):
").strip().lower()
        if retry not in ("s", "sim", "y", "yes"):
            print("Xau :)")
            break

if __name__ == "__main__":
    main()
```

Resultados dos Testes

Fórmula de Newton-Cotes Aberta com n=0 (Regra do Ponto Médio)

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 1
Insira o valor de n: 0

Aproximação do integral: 0.300558864942173
```

Fórmula de Newton-Cotes Aberta com n=1

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 1
Insira o valor de n: 1

Aproximação do integral: 0.297987542187263
```

Fórmula de Newton-Cotes Aberta com n=2

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 1
Insira o valor de n: 2

Aproximação do integral: 0.292858659192590
```

Fórmula de Newton-Cotes Aberta com n=3

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 1
Insira o valor de n: 3

Aproximação do integral: 0.292869228136084
```

Fórmula de Newton-Cotes Fechada com n = 1 (Regra do Trapézio)

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 2
Insira o valor de n: 1

Aproximação do integral: 0.277680183634898
```

Fórmula de Newton-Cotes Fechada com n = 2 (Regra de Simpson)

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 2
Insira o valor de n: 2

Aproximação do integral: 0.292932637839748
```

Fórmula de Newton-Cotes Fechada com n = 3 (Regra de Simpson dos 3/8)

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 2
Insira o valor de n: 3

Aproximação do integral: 0.292910702549171
```

Fórmula de Newton-Cotes Fechada com n = 4 (Regra de Boole)

```
Introduza a função (f(x)): sin(x)
Insira o intervalo (a b): 0 pi/4

Selecione as fórmulas a usar:
1. Fórmulas de Newton-Cotes Abertas
2. Fórmulas de Newton-Cotes Fechadas
0. Sair
Escolha (0/1/2): 2
Insira o valor de n: 4

Aproximação do integral: 0.292893182561264
```